

SmartSense Domain Model

Fifty-one entities in eleven groups, with the invariants that shape every table, the four places where no contract exists at all, and the fields we cannot finalise until a Product Owner decision lands.

DRAFTED

4 September 2026

DERIVED FROM

Contract Spine v0.6.0 · Alerts
backend spec r5 · Frontend spec
· Gateway protocol

TARGET

Serverpod on PostgreSQL

STATUS

Not approved. Blocked on B1,
B2 and B4.

APPROVED — 7 SEPTEMBER 2026

Gazihan Alankuş · GBot Software · basis for schema work.
Caveats: production cursors opaque (OQ-37, resolved);
RV Owners 'attention' sort undecided (OQ-36);
Owner Activation unmodeled until OQ-6.
Remaining: B1/B2 (OQ-34) · B4 datapoint catalogue (OQ-14).

1 Where the model comes from

51

ENTITIES

45

IN RELEASE 1

11

GROUPS

73

CONTRACT TYPES

11

INVARIANTS

4

MISSING CONTRACTS

Four sources feed this model with different standing. The **Contract Spine v0.6.0** gives 73 types covering the shapes the Portal consumes. The **alerts backend specification** gives six tables with columns, keys and constraints, and is the most precisely specified corner of the system. The **frontend specification** gives the authorization model. The **existing repositories** give the gateway parameter catalogue and the operational concepts the handoff never mentions, as evidence of what the product needed rather than as a specification.

Nothing here is a migration target. Release 1 starts with an empty database.

2 Rules that shape every table

Each is stated somewhere in the controlled material, and each constrains the schema rather than the service code. They come first because a table designed without them gets redesigned.

1 Tenant and organization are server-assigned, never parameters

No endpoint takes a tenant or organization argument. A client-supplied `organizationId` cannot select a tenant, create across tenants or transfer a record. Ownership is derived from the session's active context at write time.

2 Scope is one active organization, not the set a principal belongs to

Resolved at sign-in from the home organization, changed only through the global switcher under `tenant.switch`, audited, invalidating cached context. The server issues an opaque access-context version so a stale response cannot be reused. Never the union of the organizations a principal belongs to.

3 Timestamps and actors come from the server

Server clock, server-resolved principal. A client-supplied timestamp or actor is rejected and audited as invalid, not silently overwritten: normalising hides a caller that misunderstands the model.

4 Version rows are append-only

No UPDATE and no DELETE is ever issued against a version table. A definition or policy change appends a row and bumps a denormalised counter on the parent. Published versions are immutable, which is what makes an event traceable to the exact configuration that produced it.

5 Optimistic concurrency needs the row lock, not just the comparison

Every mutating call carries `expectedVersion`. Inside the transaction the parent row is read `FOR UPDATE` before the comparison. Comparing versions without the lock reintroduces the race it exists to close, and a conflict returns both versions with nothing written.

6 Event time and receive time are two columns, both preserved

A late arrival keeps its own event time rather than being renumbered to look prompt. Older data never becomes the current value however recently it arrived. `Replay` is deduplicated on

data never becomes the current value however recently it arrived, reply is decomposed in unit, field, event time and source. This is the only ordering rule in the product, and gateway replacement reuses it rather than defining a second one.

7 Sensitive fields are projected away at the server

A value the actor may not receive is omitted, or replaced by an explicit withheld marker, before the response leaves the server. It is never sent and hidden in the client. A withheld value is also never reconstructable from a neighbouring field, and columns whose ordering would disclose it become unsortable.

8 Refusals are audit records too

Every audited action carries a result of OK, DENIED, CONFLICT or INVALID. An audit trail that records only successes cannot answer who tried. Audit is one immutable store; organization activity, gateway history and site history are queries over it, never parallel tables.

9 The rules that matter are database constraints, not service checks

Two from the alerts spec, worth copying as a pattern. A TPMS definition applies to every Model-defined position and cannot carry one: `CHECK (NOT (applies_to_all_positions AND position IS NOT NULL))` . A configuration closure cannot impersonate a gateway clear: `CHECK (gateway_clear = false)` with `CHECK (clear_value IS NULL)` . The database refuses the state rather than the service declining to create it.

10 Forbidden and not-found look identical from outside

Authorization runs in a fixed order so an error code never reveals that a record exists in a scope the caller cannot see. Principal, then tenant, then active organization, then organization authorization, then page policy, then object resolution, then validation.

11 Undefined is a value, and inventing a range is a defect

A property the authoritative material does not define is stored and returned as not defined, never inferred to fill a column. The spine records two ranges that were removed for exactly this reason: a 1990 to 2100 model-year window and a sixty-foot length ceiling, both invented. We keep the datatype and drop the invented bound.

3 The eleven groups

Source column: `SPINE` the contract defines the shape · `SPEC` the alerts backend

specification defines it · **REPO** derived from the existing system, no contract · **DESIGN**
nothing defines it, we design it · **DEFER** registered, no Release 1 work.

GROUP	ENTITIES	SOURCE
A • Tenancy and access 9 entities	Organization, User, OrganizationMembership, UserSession, AccessProfile, CustomAccessProfile, RouteAccessPolicy, ProtectedAction, SensitiveFieldGate	SPINE
B • Product configuration 5 entities	RvModel, RvModelTirePosition, Vin, VinProductConfiguration, VinTpmsSensorAssociation	SPINE
C • Device and replacement 6 entities	Gateway, GatewayReplacementTransaction, VinContinuitySnapshot, GatewayParameterDefinition, GatewayParameterValue, GatewayCommand	SPINE REPO
D • Telemetry 4 entities	DatapointDefinition, TelemetryRecord, VinCurrentState, SignificantGEvent	SPINE
E • Alert configuration 6 entities	AlertDefinition, AlertDefinitionVersion, NotificationPolicy, NotificationPolicyVersion, AlertConfigurationClosure, AlertConfigurationAudit	SPEC
F • Alert events 2 entities	AlertEvent, AlertEventTransition	DESIGN
G • Notification delivery 4 entities	NotificationAttempt, RecipientAcknowledgement, RecipientSnooze, ReminderSchedule	DESIGN
H • Ownership 2 entities	OwnershipRecord, OwnerActivation	SPINE DESIGN
I • Geofence 5 entities	GeofenceSite, GeofenceGeometryVersion, GeofenceLocationType, GeofenceEvent, GeofenceSettings	SPINE
J • Audit 2 entities	AuditEvent, ProfileChangeEvent	SPINE
K • Deferred 6 entities	Subscription, SubscriptionPlan, Firmware, FirmwareRollout, FirmwareUpdateDevice, SimCard	DEFER

Group K is registered so the routes and policy rows have something to point at. No table in the data built in Release 1.4

it gets built in Release 1.

Two shapes in group C come only from the existing repositories.

GatewayParameterDefinition is the 117-row protocol catalogue, and GatewayParameterValue is the desired-versus-device-reported pair that produces the Configuration column's Current, Pending, Drift and Failed states on Fleet. The handoff describes those states without ever saying where they come from; the answer is in the previous build, and it is correct.

4 Four contracts that do not exist

THE GAP THIS DRAFT SURFACES

The alerts backend specification says, of the shared-TPMS rule, that "Alert Event rows keep the triggering VIN, tire position, sensor identity and Gateway" and that "no change to event storage is required". It is describing an event store as though one is specified. None is. Not in the Contract Spine, not in the alerts specification, not in the notifications handoff.

The same is true of the notification delivery model, the principal and session model, and Owner Activation as an independent state. Four things the product cannot work without, defined nowhere.

F • The alert event

Everything the Portal shows on the Alerts destination and on Alert Event Detail reads from this, and nothing specifies it. What we know it must carry, assembled from constraints scattered across four documents: gateway-assigned event identity preserved end to end, the triggering VIN and tire position, the sensor and gateway that produced it, the measured value and unit, the definition and the exact definition version in effect when it fired, severity as supplied by the server and never derived, and a state that distinguishes Active, Gateway Clear and Definition Disabled without letting the last impersonate the second.

It must also carry event time and receive time separately, per invariant 6, and it must survive its source gateway being replaced. Significant-G is a momentary event with no return-to-normal, which is why we hold it as its own entity rather than as an alert event with a null clear.

G • Notification delivery

Document 04 specifies the behaviour precisely and the storage not at all. Recipients

resolve to fixed Release 1 classes, Primary Owner plus eligible Authorized Users, so a recipient is resolved per event rather than stored on a policy. Acknowledgement stops reminders for one recipient only. Snooze pauses them for one recipient only, in whole positive minutes, and expires. Actual attempts and their outcomes are evidence, and a failed provider attempt is visible as failed rather than counted as delivered.

That last sentence is a schema requirement, not a service one. A boolean flag on the event cannot express it, which is precisely how the previous build came to record a failed push as delivered.

A • Principal and session

The spine describes `PortalSessionContext` as a projection but declares no user entity. We need the principal, account state across Active, Invited, Suspended and Deactivated, the organization membership that makes a user reachable in more than one tenant, the session with its opaque access-context version, and the mobile device registration that push delivery resolves against.

H • Owner Activation

The live review recorded this as requiring reconciliation and it is still open. The controlled material makes Owner Activation a distinct fact with an immutable original activation date. The prototype derives activation from the claim state, so the two can never disagree, which means the prototype cannot demonstrate the case the requirement exists for. Making it real needs a backend definition, a mobile contract for the activation step and a Portal field. This is question N7 and we should not model it until it is answered.

5 Model sketches

Four sketches, chosen because their invariants are the ones most easily lost in translation. Serverpod syntax here is indicative; exact relation and scope syntax gets confirmed against the generator when we scaffold.

Alert definition and its append-only versions

Straight from the specification. The parent holds the denormalised current version so the concurrency check needs one row lock, and the version table is the only place configuration lives.

```

# alert_definition.spy.yaml
class: AlertDefinition
table: alert_definition
fields:
  publicId: String          # AD-0001, unique per organization
  tenantId: String          # server-assigned from session, never a parameter
  organizationId: String    # server-assigned from session
  policyId: String          # 1:1 with the dedicated notification policy
  createdAt: DateTime       # server clock
  createdBy: String         # server-resolved principal
  currentDefinitionVersion: int
  currentPolicyVersion: int
indexes:
  alert_definition_org_public_idx:
    fields: organizationId, publicId
    unique: true
  alert_definition_tenant_idx:
    fields: tenantId, organizationId

# alert_definition_version.spy.yaml – APPEND ONLY. No update, no delete.
class: AlertDefinitionVersion
table: alert_definition_version
fields:
  definitionId: String
  version: int
  savedAt: DateTime         # server clock
  savedBy: String           # server-resolved
  change: String?
  name: String
  description: String?
  enabled: bool
  measurementId: String     # must be alert-eligible in DatapointDefinition
  gatewayMapping: String?
  gatewaySettingId: int?    # 831/832/833 pressure, 815/816/817 temperature
  operator: AlertOperator
  threshold: double
  unit: String
  severity: AlertSeverity   # Advisory | Warning | Urgent, server-authoritative
  returnToNormal: String?  # absent for a momentary measurement
  appliesToAllPositions: bool
  position: String?        # NULL-only when appliesToAllPositions
indexes:
  alert_definition_version_pk_idx:
    fields: definitionId, version
    unique: true

```

Two constraints go into the migration by hand, because they are the model rather than decoration:

```

ALTER TABLE alert_definition_version ADD CONSTRAINT tpms_shared_threshold_has_no_position
    CHECK (NOT (applies_to_all_positions AND position IS NOT NULL));

-- a configuration closure can never look like a gateway clear
ALTER TABLE alert_configuration_closure ADD CONSTRAINT closure_is_not_a_gateway_clear
    CHECK (gateway_clear = false AND clear_value IS NULL);

```

Telemetry, with the two time columns

The dedup index is the replay rule made structural. Current state is a separate projection precisely so that a late arrival cannot overwrite it.

```

class: TelemetryRecord
table: telemetry_record
fields:
  vinId: String
  fieldId: String          # DatapointDefinition.defId
  value: String
  eventTimeUtc: DateTime   # from the source. Required.
  receivedAtUtc: DateTime  # its own fact. Never backfilled from eventTime.
  sourceGatewayId: String? # may since have become Replaced
indexes:
  telemetry_replay_dedup_idx:
    fields: vinId, fieldId, eventTimeUtc, sourceGatewayId
    unique: true          # a buffered upload does not multiply the evidence
  telemetry_history_idx:
    fields: vinId, fieldId, eventTimeUtc

```

Tire position, where the SSP lives

A position list is never generated from a tire count, and a pressure is never held once for the whole model. The canonical names and aliases are a closed set of six.

```

class: RvModelTirePosition
table: rv_model_tire_position
fields:
  modelId: String
  positionId: String      # TPMS_1 ... TPMS_6, device-facing
  sequence: int           # front to rear
  canonicalName: String   # Off Door Side Front ... Door Side Rear
  displayAlias: String    # ODSF DSF ODSM DSM ODSR DSR – no other alias exists
  axleIndex: int          # 1, 2 or 3
  side: TireSide          # DOOR | OFF_DOOR

```



```

    specifiedServicePressurePsi: double    # SSP. No default, no template value.
indexes:
  rv_model_tire_position_idx:
    fields: modelId, positionId
    unique: true

```

Three things about that last field. It is a double and not an int, because the runtime accepts and persists 65.5 psi. It has no default, because a layout template carries no pressure and every configured position must be given one explicitly. And it is the manufacturer's recommended cold pressure, not a measured value and not an alert threshold, so nothing in the alerts model may read it as one.

The gateway replacement transaction

State is derived from the two readiness gates on every mutation, so a client cannot post a readiness it has not earned. The old device stays Current until an atomic cutover, and continuity is measured rather than asserted: the snapshot is taken at start, read again after cutover, and equality is the evidence.

```

class: GatewayReplacementTransaction
table: gateway_replacement_transaction
fields:
  vinId: String
  organizationId: String      # server-derived from the unit, never supplied
  state: ReplacementState    # Started | SetupInProgress | ReadyForCutover
                                # | Complete | Aborted – DERIVED, never an input
  outgoingGatewayId: String
  replacementGatewayId: String
  configurationApplied: bool
  requiredPositionIds: List<String>  # copied at start; a later Model edit
                                      # cannot redefine the requirement
  pairedPositionIds: List<String>
  startedBy: String
  startedAtUtc: DateTime
  closedAtUtc: DateTime?
  abortReason: String?
  continuityAtStart: VinContinuitySnapshot
  continuityAtCutover: VinContinuitySnapshot?
indexes:
  one_open_transaction_per_vin_idx:
    fields: vinId
    unique: true              # partial index on open states in the migration

```

The unique index needs a WHERE clause on the pre-cutover states, added in the migration, so a VIN can have many closed transactions and only one open one. Two concurrent jobs

on one vehicle is exactly the half-replaced state the transaction exists to prevent.

6 Fields blocked on a decision

Six open Product Owner decisions map to specific columns. In every case the datatype is settled and only the constraint is open, so we can build now and add the constraint later without a migration that touches data. What we must not do is invent the bound, which is how the removed model-year window and length ceiling got there.

FIELD	WHAT IS OPEN	OUR POSITION
<code>RvModelTirePosition.specifiedServicePressurePsi</code>	Allowable range, precision and permitted decimal step. Nothing establishes a minimum, a maximum or a step.	Store numeric, enforce positive only
<code>RvModel.modelYear</code>	Allowable range. The invented 1990 to 2100 window was removed.	Datatype only
<code>RvModel.lengthFeet</code>	Allowable range and precision. The invented sixty-foot ceiling was removed.	Positive only
<code>RvModel.rvClass</code>	No controlled document defines the enumeration.	Free text until defined, not an enum
<code>Vin.includedTermMonths</code>	Permitted range or list. 36 months is the controlled default; nothing establishes the allowed values or a maximum.	Store the snapshot, no validation
Model revision behaviour	What happens to units in the field when an SSP changes on a Model already in use. No controlled revision or deployment semantics exist.	Protect the change, do not propagate
AuditEvent on denied pairing	Whether every authorization denial must produce an audit record, or only successful association and the cross-VIN refusal.	Audit denials, flag as our choice

AND ONE THAT BLOCKS THE WHOLE ALERTS GROUP

Question B4. The production datapoint catalogue is deliberately empty and the prototype's eight measurements are declared review-only. `AlertDefinitionVersion` `measurementId` is

eight measurements are declared review only. Alert level and alert severity are not a foreign key into a catalogue nobody has authored, and the service must validate that a measurement is alert-eligible, that the operator is one it declares, that the threshold is within its bounds and that the unit is one it permits. None of that can be built against an empty catalogue. Group E is designable now and testable only after B4.

7 Serverpod notes

Four decisions worth making once, at the start, rather than discovering across forty model files.

- **Serialize domain enums by name.** Index-based serialization is the default and it means reordering an enum silently rewrites the meaning of stored rows. Severity, replacement state, claim state and alert state are all persisted and all likely to gain values, so every one of them declares `serialized: byName`.
- **Tenant scope belongs to the session helper, not to each endpoint.** One resolver returns the principal, the tenant and the active organization, and every query builds on it. The previous backend dispatched scope on model class name through a long if-else chain that fell through to an empty result for anything newly added; correct by accident is not a pattern to repeat.
- **The four authorization layers resolve in one place.** Page policy, protected action, field projection and object scope are separate decisions on every request. We want a single guard that takes the route id and the action, not four checks copy-pasted into each endpoint. This is one of the three phase 0 spikes.
- **Append-only tables get no generated update path.** Serverpod will happily generate one. For the version and audit tables we wrap them behind a repository that exposes append and read only, so nothing can call an update that the model says does not exist.

One more, on the ingestion side rather than the model: telemetry arrives over MQTT from AWS IoT Core, and in the previous build the listener ran unsupervised inside the web container. It belongs in its own service with its own scaling and restart policy, writing through the same dedup index as everything else.

8 Build order

Groups A, B and J first, since tenancy, access, product configuration and audit are what everything else references. Then F and G once the event and delivery contracts are approved. Then C and D with the ingestion service, once the gateway protocol questions land. Then E, which is designable now and testable only once the measurement catalogue exists. H and I last.

This sits inside the development order in document 00 §5, which we follow rather than replace.